MNS University OF Agriculture Multan

Vehicle Service Center

Database Management System

Project Report

GROUP 4 — TEAM MEMBERS
Muhmmad Zubair | Admin / Project Lead
Abubakar Ilyas | Database Developer
Adil Shahzad | Database Developer
Tauseef Subhani | Database Developer
Noman | Database Developer

Submitted To:	Mam Sania Ayoub
Subject	Database Management Systems
Tool	MySQL 8.0 + MySQL Workbench
Submitted	08-June-2026

Table of Contents

1. Introduction

This report presents the design, implementation, and functionality of the **Vehicle Service Center Database Management System**, developed as part of a university Database Management Systems course. The system is built entirely in MySQL 8.0 and is intended to manage the complete operational workflow of an automobile service center.

Modern vehicle service centers handle hundreds of customer visits, vehicle records, parts transactions, and employee assignments daily. Without a structured database, this information becomes difficult to track, report on, or maintain accurately. This project addresses that need by providing a fully normalized relational database with stored procedures, views, and indexes that support efficient data management and reporting.

2. Project Objectives

- Design a fully normalized (3NF) relational database for a vehicle service center.
- Manage customer profiles and their associated vehicle records.
- Create and track job cards from vehicle intake through service completion.

- Maintain a parts inventory with automatic stock deduction and reorder alerts.
- Record service history for every vehicle and generate performance reports.
- Implement stored procedures and functions to encapsulate business logic.
- Apply indexing strategies to optimize query performance.
- Demonstrate practical use of MySQL views for reporting.

3. System Overview

The system is delivered as a single self-contained SQL script (**vehicle_service_center.sql**) that, when executed, creates the database, all tables, inserts sample data, creates views, and registers stored procedures and functions. No external application or frontend is required — everything runs inside MySQL Workbench or the MySQL command-line client.

Technology Stack

Component	Details
DBMS	MySQL 8.0+
Storage Engine	InnoDB (all tables — full FK and transaction support)
Character Set	utf8mb4 / utf8mb4_unicode_ci
Normalization	Third Normal Form (3NF)
IDE / Tool	MySQL Workbench 8.0+
Script File	vehicle_service_center.sql (738 lines)

4. Database Design & Normalization

Normalization

Normal Form	Rule Applied	Example
1NF	All columns are atomic; no repeating groups	Separate vehicle_makes and vehicle_models tables
2NF	No partial dependency on a composite key	Every Tabe Work Properly
3NF	No transitive dependency	

Design Decisions

- **Lookup tables** for vehicle_makes and vehicle_models avoid repeating brand/model strings across thousands of vehicle records.
- **Computed column** grand_total in job_cards is a GENERATED ALWAYS AS column — it is always accurate and never needs manual updating.
- **InnoDB engine** enforces referential integrity through foreign keys with ON DELETE CASCADE and ON DELETE RESTRICT rules.
- **ENUM types** are used for job status and employee roles to constrain values at the database level.

• **UNSIGNED integers** are used for all ID and quantity columns to prevent negative values.

5. Entity Relationship Diagram

The following diagram shows all 10 tables and their relationships. Primary keys (PK) and foreign keys (FK) are marked on each entity.

Table	Primary Key	Foreign Keys	Relationship
customers	customer_id	—	1 customer → many vehicles
vehicle_makes	make_id	—	1 make → many models
vehicle_models	model_id	make_id → vehicle_makes	1 model → many vehicles
vehicles	vehicle_id	customer_id, model_id	1 vehicle → many job cards
employees	employee_id	—	1 employee → many jobs/services
service_types	service_type_id	—	1 type → many job services
parts	part_id	—	1 part → many job parts
job_cards	job_id	vehicle_id, advisor_id	1 job → many services & parts
job_services	job_service_id	job_id, service_type_id, mechanic_id	Links job ↔ service ↔ mechanic
job_parts	job_part_id	job_id, part_id	Links job ↔ parts used

To view the full graphical ER Diagram: open MySQL Workbench → Database → Reverse Engineer → select vehicle_service_center

Relationship Summary

```
customers ||--o{ vehicles (one customer owns many vehicles)
vehicle_makes ||--o{ vehicle_models (one make has many models)
vehicle_models ||--o{ vehicles (one model categorizes many vehicles)
vehicles ||--o{ job_cards (one vehicle generates many job cards)
employees ||--o{ job_cards (one advisor handles many job cards)
job_cards ||--o{ job_services (one job card includes many services)
job_cards ||--o{ job_parts (one job card uses many parts)
service_types ||--o{ job_services (one service type performed in many jobs)
parts ||--o{ job_parts (one part issued across many job records)
employees ||--o{ job_services (one mechanic performs many services)
```

6. Table Descriptions

Table: customers

Stores master data for all customers.

Column	Type	Constraint	Description

customer_id	INT UNSIGNED	PK, AUTO_INCREMENT	Unique customer identifier
full_name	VARCHAR(100)	NOT NULL	Customer full name
phone	VARCHAR(20)	NOT NULL, UNIQUE	Mobile number (unique)
email	VARCHAR(120)	UNIQUE	Email address
address	VARCHAR(255)	—	Street address
city	VARCHAR(60)	—	City
created_at	TIMESTAMP	DEFAULT NOW()	Record creation time

Table: vehicles

Records each vehicle linked to a customer and model.

Column	Type	Constraint	Description
vehicle_id	INT UNSIGNED	PK	Unique vehicle identifier
customer_id	INT UNSIGNED	FK	References customers
model_id	SMALLINT UNSIGNED	FK	References vehicle_models
year	YEAR	NOT NULL	Manufacturing year
reg_number	VARCHAR(20)	UNIQUE	Registration number
color	VARCHAR(30)	—	Vehicle color
engine_cc	SMALLINT UNSIGNED	—	Engine displacement
mileage_km	INT UNSIGNED	DEFAULT 0	Current mileage in km

Table: job_cards

One record per workshop visit. Central table linking vehicles, advisors, services, and parts.

Column	Type	Constraint	Description
job_id	INT UNSIGNED	PK	Unique job identifier
vehicle_id	INT UNSIGNED	FK	Vehicle being serviced
advisor_id	INT UNSIGNED	FK	Service advisor assigned
date_in	DATETIME	DEFAULT NOW()	Vehicle intake date/time
date_out	DATETIME	NULLABLE	Completion date/time
status	ENUM	NOT NULL	Open / In Progress / Completed / Cancelled
grand_total	DECIMAL(10,2)	GENERATED	labor + parts - discount (auto-computed)

Table: parts

Inventory of all spare parts with pricing and stock levels.

Column	Type	Constraint	Description

part_id	INT UNSIGNED	PK	Unique part identifier
part_name	VARCHAR(120)	NOT NULL	Descriptive name
part_number	VARCHAR(50)	UNIQUE	SKU / part number
brand	VARCHAR(60)	—	Manufacturer brand
cost_price	DECIMAL(10,2)	NOT NULL	Purchase cost
selling_price	DECIMAL(10,2)	NOT NULL	Retail selling price
stock_qty	INT UNSIGNED	NOT NULL	Current stock quantity
reorder_level	INT UNSIGNED	NOT NULL	Alert threshold for restocking

Remaining Tables Summary

Table	Purpose	Key Columns
vehicle_makes	Lookup for car brands	make_id PK, make_name UNIQUE
vehicle_models	Lookup for models per brand	model_id PK, make_id FK, model_name
employees	All staff — mechanics, advisors, managers	employee_id PK, role ENUM, salary
service_types	Service catalogue with standard durations	service_type_id PK, base_labor_cost
job_services	Services performed on a job card	job_id FK, service_type_id FK, mechanic

Table	Purpose	Key Columns
job_parts	Parts issued on a job card	job_id FK, part_id FK, qty_used, unit_price

In addition to the automatically created primary key indexes and foreign key indexes, the following explicit indexes were created to optimize the most common query patterns:

Index Name	Table	Column(s)	Query Optimized
idx_vehicles_reg	vehicles	reg_number	Search vehicle by registration plate
idx_vehicles_customer	vehicles	customer_id	List all vehicles for a customer
idx_job_cards_status	job_cards	status	Filter Open / In Progress jobs
idx_job_cards_date_in	job_cards	date_in	Date range revenue / history queries
idx_parts_stock	parts	stock_qty	Low-stock inventory alerts
idx_parts_number	parts	part_number	Look up a part by SKU
idx_customers_phone	customers	phone	Customer lookup by phone number
idx_job_services_job	job_services	job_id	Fetch all services for a job card

All indexes use B-Tree structure (MySQL default), which provides O(log n) lookup performance for equality and range queries. Composite indexes were deliberately avoided in this design as the single-column indexes cover all identified query patterns without the overhead of wider index entries.

Five views are defined to simplify reporting and data access. Each view encapsulates a complex multi-table JOIN so that reports can be generated with a single SELECT statement.

vw_job_summary

Combines job_cards, vehicles, customers, vehicle_makes, vehicle_models, and employees into a single flat row per job. Used for front-desk job tracking and billing.

Usage:

```
SELECT * FROM vw_job_summary WHERE status = 'In Progress';
```

vw_low_stock_parts

Filters the parts table to show only items at or below their reorder_level. Includes a shortage column showing how many units are needed.

Usage:

```
SELECT * FROM vw_low_stock_parts;
```

vw_monthly_revenue

Aggregates completed job cards by month, showing total jobs, total revenue, average job value, and total discounts given.

Usage:

```
SELECT * FROM vw_monthly_revenue;
```

vw_mechanic_performance

Summarizes services performed and total labor earned per mechanic. Used for payroll verification and performance reviews.

Usage:

```
SELECT * FROM vw_mechanic_performance ORDER BY total_labor_earned DESC;
```

vw_vehicle_history

Shows the complete service history for every vehicle — all job cards, services performed, and amounts charged.

Usage:

```
SELECT * FROM vw_vehicle_history WHERE reg_number = 'LHR-2019-001';
```

All business logic is encapsulated in stored procedures and functions, keeping the application layer thin and ensuring data integrity rules are enforced at the database level.

Stored Procedures

Procedure	Parameters	Description
sp_get_customer_profile	customer_id INT	Returns customer info + all vehicles
sp_create_job_card	vehicle_id, advisor_id, complaint	Opens a new job card with status Open
sp_add_service_to_job	job_id, service_type_id, mechanic	Abor_cost, notAdds service and auto-updates
sp_issue_part	job_id, part_id, qty	Issues part, validates stock, updates inventory and total
sp_complete_job	job_id, discount DECIMAL	Closes job card, sets date_out, applies discount
sp_search_vehicle	keyword VARCHAR	Search by registration number or customer name
sp_top_customers	limit INT	Returns top N customers ranked by total spend
sp_restock_part	part_id INT, qty_add INT	Increases stock quantity for a part

Function	Parameters	Returns	Description
fn_revenue_in_range	start DATE, end DATE	DECIMAL(12,2)	Completly fill all the data.
fn_vehicle_job_count	vehicle_id INT	INT	Count of all job cards for a specific

Functions

Key Procedure Logic: sp_issue_part

This procedure demonstrates the most critical business rule in the system. Before issuing a part it validates that sufficient stock is available, raises an SQL error (SQLSTATE 45000) if not, then decrements the stock and recalculates the job's parts total in a single atomic operation.

- Step 1: Read current stock_qty and selling_price for the requested part.
- Step 2: IF stock_qty < qty_requested THEN raise SIGNAL error — transaction stops.
- Step 3: INSERT a new row into job_parts with the issued quantity and price.
- Step 4: UPDATE parts SET stock_qty = stock_qty - qty (atomic decrement).
- Step 5: UPDATE job_cards.total_parts by re-summing all job_parts for this job.
- Step 6: Return rows_affected and remaining_stock as confirmation.

The script inserts a medium-sized realistic dataset representing approximately 3–4 months of operations at a busy urban service center.

Table	Rows Inserted	Notable Data
customers	25	25 customers from 8 cities across Pakistan
vehicle_makes	10	Toyota, Honda, Suzuki, Hyundai, KIA, Nissan, Mitsubishi, Daihatsu, Changan,
vehicle_models	23	Corolla, Civic, Alto, Tucson, Sportage, Swift, Lancer, Hilux, and more
vehicles	25	Model years 2015–2023, mileage from 3,000 to 130,000 km
employees	10	4 mechanics, 2 advisors, 2 supervisors, 1 parts manager

service_types	15	Oil change, brake pads, AC recharge, full service, gearbox, etc.
parts	25	Engine oils, filters, batteries, brake pads, spark plugs, belts, and more
job_cards	25	24 completed, 1 in progress; date range Jan–May 2024
job_services	32	Multiple services per job card; covers all 15 service types
job_parts	30	Parts matched to jobs; quantities and prices reflecting real usage

Financial Summary of Sample Data

Metric	Value
Total job cards	25
Completed jobs	24
In-progress jobs	1
Total revenue (Q1 2024)	PKR 184,350
Total revenue (Q2 2024)	PKR 181,000 (approx.)
Largest single job	PKR 14,800 (Full major service)
Smallest single job	PKR 800 (Oil change)
Total discounts given	PKR 2,700
Parts at or below reorder level	3 parts

Query 1: All job cards with customer and vehicle info

```
SELECT * FROM vw_job_summary ORDER BY job_id;
```

Expected Output: *Returns 25 rows showing customer name, vehicle, advisor, date in/out, status, labor, parts, discount, and grand total.*

Query 2: Monthly revenue report

```
SELECT * FROM vw_monthly_revenue;
```

Expected Output: *Returns one row per month (Jan–May 2024) with job count, total revenue, average job value, and total discounts.*

Query 3: Top 5 spending customers

```
CALL sp_top_customers(5);
```

Expected Output: *Returns the 5 customers with the highest cumulative spend, including total jobs and last visit date.*

Query 4: Low stock parts alert

```
SELECT * FROM vw_low_stock_parts;
```

Expected Output: *Returns parts where stock_qty <= reorder_level, showing the shortage quantity.*

Query 5: Search vehicle by registration

```
CALL sp_search_vehicle('LHR');
```

Expected Output: *Returns all vehicles with registration numbers containing 'LHR' and their owner details.*

Query 6: Revenue for H1 2024

```
SELECT fn_revenue_in_range('2024-01-01','2024-06-30') AS h1_revenue;
```

Expected Output: *Returns a single decimal value representing total completed job revenue in the date range.*

Query 7: Issue a part from inventory

```
CALL sp_issue_part(1, 10, 4);
```

Expected Output: *Issues 4 spark plugs (part_id=10) to job_id=1. Returns rows_affected=1 and remaining stock count.*

Query 8: Mechanic performance report

```
SELECT * FROM vw_mechanic_performance ORDER BY total_labor_earned DESC;
```

Expected Output: *Lists each mechanic with services count and total labor revenue earned.*

Method 1 — MySQL Workbench (Recommended)

- Step 1: Install MySQL 8.0+ and MySQL Workbench 8.0+ if not already installed.
- Step 2: Open MySQL Workbench and click your local connection (root @ localhost:3306).
- Step 3: Enter the root password when prompted.
- Step 4: Go to File → Open SQL Script… and select vehicle_service_center.sql.
- Step 5: Press Ctrl + Shift + Enter to execute the entire script.
- Step 6: Check the Output pane at the bottom for green checkmarks.
- Step 7: Expand Schemas → vehicle_service_center in the left panel to browse all 10 tables.
- Step 8: Scroll to the bottom of the script, click any SELECT query, and press Ctrl + Enter to run it.

Method 2 — Command Line (CLI)

```
mysql -u root -p < vehicle_service_center.sql
```

After running, verify with:

```
mysql -u root -p vehicle_service_center

SHOW TABLES;

SELECT COUNT(*) FROM customers; -- Should return 25
```

Generate the ER Diagram in Workbench

- Step 1: After running the SQL script, go to Database → Reverse Engineer… in the menu.
- Step 2: Select your connection and click Next.
- Step 3: Select the vehicle_service_center schema and click Next.
- Step 4: Workbench will auto-generate the full graphical ER diagram.
- Step 5: Go to File → Save Model to save it as vehicle_service_center.mwb.

Troubleshooting

Error	Cause	Fix
Database already exists	Script run twice	Run: DROP DATABASE vehicle_service_center
DELIMITER not supported	Used Ctrl+Enter (single stmt)	Use Ctrl+Shift+Enter (Execute All) instead
Access denied for root	Wrong password or socket	Add --host=127.0.0.1 --port=3306 to CLI command
mysql not found (Windows CLI)	MySQL not in system PATH	Navigate to C:\Program Files\MySQL\...\bin\ first

This project successfully demonstrates the design and implementation of a complete, production-ready relational database system for a vehicle service center. The database covers all core operational requirements: customer and vehicle management, job card lifecycle, parts inventory with automatic stock control, employee assignment, and financial reporting.

The design strictly follows Third Normal Form, eliminating data redundancy and ensuring data integrity through foreign key constraints, ENUM types, and computed columns. Performance is addressed through eight targeted indexes. Business logic is encapsulated in eight stored procedures and two functions, making the database self-contained and easily integrable with any frontend application in the future.

The five views provide instant reporting capability without requiring complex queries from the application layer. The medium-sized sample dataset (25 customers, 25 vehicles, 25 job cards, 32 services, 30 parts transactions) provides realistic test data for all functionality.

Submitted Files

File	Description
vehicle_service_center.sql	Complete MySQL script — 738 lines — all tables, data, views, procedures
README.md	Project documentation with setup instructions and query reference
Vehicle_Service_Center_Report.pdf	This project report
vehicle_service_center.mwb	MySQL Workbench ER Diagram model (generated via Reverse Engineer)

Group 4 | Vehicle Service Center Database | Database Management Systems Course